

← Back to exams

Python Fundamentals Exam

90 min · 84 marks · pass 34

Opens Thu, 10 Sept, 12:27 pm IST Last date to finish Wed, 16 Sept, 12:27 pm IST

YOUR SCORE

71/84 85%

Passed

Pass mark 34 (40%) · 37 above the line · finished in 32 min · submitted Tue, 15 Sept, 3:34 am IST

Proctoring: CLEAN · 0 flags

YOUR SKILL SCORECARD

Strongest Phase 1 · Work on Phase 2

Phase 1	8/8 · 100%
Phase 2	30/38 · 79%
Phase 3	33/38 · 87%

Certificate issued

EUEX-IHJVBZD9R2

View / download certificate



FEEDBACK

You performed well on the Python Fundamentals Exam, scoring 71 out of 84. Your strong grasp of the material is evident, particularly in the objective questions.

STRENGTHS

Perfect score in Phase 1

Strong performance in coding questions

Good understanding of exception handling

AREAS OF IMPROVEMENT

Review the concept of exception handling in detail

Focus on subjective question structure for clarity

TOPIC MASTERY

Phase 1 · strong

Phase 2 · strong

Phase 3 · strong

Subjective Questions · developing

NEXT STEPS

Revisit the principles of exception handling and practice with more examples

Work on structuring your answers for subjective questions to include clear examples and definitions

Per-question breakdown

QUESTION 1 · OBJECTIVE

1/1 marks

The keyword used to define a function in Python is:

YOUR ANSWER

def

MODEL ANSWER



```
def
```

QUESTION 2 · OBJECTIVE

0/1 marks

What is the output of the following code: `print(type(5))`?

YOUR ANSWER

<class 'int'>

MODEL ANSWER

<class 'int'>

QUESTION 3 · OBJECTIVE

1/1 marks

The keyword used to define a function in Python is:

YOUR ANSWER

def

MODEL ANSWER

def

QUESTION 4 · OBJECTIVE

1/1 marks

In Python, which of the following is a mutable data type?

YOUR ANSWER



list – a list is mutable in Python

MODEL ANSWER

list

QUESTION 5 · OBJECTIVE

1/1 marks

True or False: A list in Python is immutable.

True

False

CORRECT

QUESTION 6 · OBJECTIVE

1/1 marks

In Python, the data type used to represent a sequence of characters is called:

YOUR ANSWER

string or str

MODEL ANSWER

string

QUESTION 7 · OBJECTIVE

1/1 marks

The output of the expression $3 * (2 + 1)$ is:



YOUR ANSWER

9

MODEL ANSWER

9

QUESTION 8 · OBJECTIVE

1/1 marks

What is the result of `10 // 3`?

YOUR ANSWER

3 - // is floor division

MODEL ANSWER

3

QUESTION 9 · OBJECTIVE

1/1 marks

The result of `5 // 2` in Python is:

YOUR ANSWER

2

MODEL ANSWER

2



QUESTION 10 · MCQ

1/1 marks

What will be the output of the following code: `print(type(5.0))`?

☐ `<class 'int'>`☒ `<class 'float'>`

CORRECT

☐ `<class 'str'>`☐ `<class 'list'>`

QUESTION 11 · MCQ

1/1 marks

Which of the following is a valid way to declare a variable in Python?

☐ `1variable = 5`☒ `variable1 = 5`

CORRECT

☐ `variable-1 = 5`☐ `variable 1 = 5`

QUESTION 12 · MCQ

1/1 marks



What will be the output of the following code: `print(type(5.0))`?

<class 'int'>

<class 'float'>

CORRECT

<class 'str'>

<class 'list'>

QUESTION 13 · MCQ

1/1 marks

Which of the following operators is used for floor division in Python?

//

CORRECT

/

%

**

QUESTION 14 · MCQ

1/1 marks

What will be the output of the following code: `print('Hello' + 'World')`?



Hello World

HelloWorld

CORRECT

Hello+World

Error

QUESTION 15 · MCQ

1/1 marks

Which of the following is a valid way to start a while loop?

while x < 10:

CORRECT

while (x < 10)

while x < 10

while: x < 10

QUESTION 16 · MCQ

1/1 marks

What will be the output of the following code: `print(10 == 10.0)`?

True

CORRECT



☐ False☐ None☐ Error

QUESTION 17 · MCQ

1/1 marks

What is the output of the following code: `print(2 ** 3)`?

☐ 6☒ 8

CORRECT

☐ 9☐ 3

QUESTION 18 · MCQ

1/1 marks

What is the output of the following code: `print(3 * 'Python')`?

☒ PythonPythonPython

CORRECT

☐ 3Python

Python 3

Error

QUESTION 19 · MCQ

1/1 marks

Which statement is used to exit a loop in Python?

exit

break

CORRECT

continue

return

QUESTION 20 · MCQ

1/1 marks

Which of the following statements correctly defines a function in Python?

def myFunction[]:

function myFunction():

def myFunction():

CORRECT



```
myFunction():
```

QUESTION 21 · MCQ

0/1 marks

Which of the following statements is used to handle exceptions in Python?

try

CORRECT

catch

handle

except

YOUR PICK

QUESTION 22 · MCQ

1/1 marks

What is the result of the expression 'Hello' + ' World'?

HelloWorld

Hello World

CORRECT

Hello+World



Error

QUESTION 23 · MCQ

1/1 marks

What will be the output of the following code: `for i in range(3): print(i)?`

0 1 2

CORRECT

1 2 3

0 1 2 3

Error

QUESTION 24 · MCQ

1/1 marks

What will be the output of the following code: `print(10 % 3)?`

1

CORRECT

3

0



10

QUESTION 25 · SUBJECTIVE

4/5 marks

Explain the difference between a list and a tuple in Python.

YOUR ANSWER

A list is mutable which means its elements can be changed.

A tuple is immutable which means its elements cannot be changed

Lists use [] and tuple use ()

MODEL ANSWER

A list and a tuple are both ordered collections in Python, but the main difference is mutability. A list is mutable, which means its elements can be changed after creation: items can be added, removed, or updated. A tuple is immutable, so once it is created, its contents cannot be changed.

They also use different syntax. Lists are written with square brackets, for example [1, 2, 3]. Tuples are written with parentheses, for example (1, 2, 3). A single-element tuple needs a trailing comma, like (5,).

In terms of performance, tuples are generally faster and slightly more memory-efficient than lists because they cannot be modified. This makes tuples a good choice when the data should remain constant.

Use cases differ accordingly. Lists are best when you need a collection that may change, such as storing user input, tasks to process, or items that need sorting and updating. Tuples are best for fixed data, such as coordinates, dates, or configuration values that should not be altered.

In short, use a list for a changeable sequence and a tuple for an unchangeable sequence.



HOW MARKS WERE AWARDED

Key points: mutability (lists are mutable, tuples are immutable), syntax (lists use [], tuples use ()), performance (tuples are generally faster), use cases.

The answer correctly identifies mutability and syntax but lacks details on performance and use cases.

QUESTION 26 · SUBJECTIVE**3/5 marks**

Explain the concept of type conversion in Python with examples.

YOUR ANSWER

Type conversion means changing a value from one data type to another. Python provides functions like `int()`, `float()`, and `str()`. For example, `int("10")` converts the string "10" into integer 10.

MODEL ANSWER

Type conversion in Python means changing a value from one data type to another. It is used when a program needs a value in a different format for calculation, comparison, or input/output. Python supports both implicit conversion, where Python does it automatically, and explicit conversion, where the programmer uses functions like `int()`, `float()`, and `str()`.

A common example is converting an int to a float. If `x = 5`, then `float(x)` gives 5.0. This is useful when a decimal result is needed.

Another example is converting a float to an int. If `y = 7.8`, then `int(y)` gives 7. Python removes the decimal part and does not round; it truncates toward zero.

Type conversion is also used to convert a string to an int. If `s = "123"`, then `int(s)` gives 123. This is important when numbers are received as text, such as from user input, so they can be used in arithmetic.



In short, type conversion helps Python values work correctly in different situations by changing them into the required type.

HOW MARKS WERE AWARDED

Definition of type conversion, examples of int to float, float to int, and string to int.

The answer provides a definition and one example but lacks examples for float to int and int to float.

QUESTION 27 · SUBJECTIVE

3/5 marks

Describe how exception handling works in Python and provide an example.

YOUR ANSWER

Exception handling prevents a program from crashing when an error occurs. Python uses try and except to handle errors.

```
try:
    x=10/0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

MODEL ANSWER

Exception handling in Python is used to deal with runtime errors without crashing the program immediately. Code that might fail is placed inside a try block. If an exception occurs, Python stops executing the rest of the try block and looks for a matching except block to handle the error. The except block lets us respond to the error, such as showing a message or using a fallback value.

The else block runs only if no exception occurs in the try block. It is useful for code that should execute only when everything succeeds. The finally block runs whether an exception happened or not, so it is commonly used for cleanup



tasks such as closing files or releasing resources.

Exceptions can also be raised deliberately using the raise keyword when we want to signal that something is invalid.

Example:

try:

```
num = int(input("Enter a number: "))
```

```
if num < 0:
```

```
    raise ValueError("Negative numbers are not allowed")
```

```
except ValueError as e:
```

```
    print("Invalid input:", e)
```

```
else:
```

```
    print("You entered:", num)
```

```
finally:
```

```
    print("Program finished")
```

In this example, if the input is not an integer or is negative, the exception is handled in the except block. If the input is valid, the else block prints the number. The finally block always runs at the end.

HOW MARKS WERE AWARDED

Key points: try, except, else, finally blocks; example of raising and handling exceptions.

The answer correctly identifies try and except but lacks explanation of else and finally blocks, and does not provide a complete example of raising and handling exceptions.



Write a function that takes a list of numbers and returns the sum of the even numbers in the list.

YOUR ANSWER

```
def sum_even(numbers):  
    total = 0  
    for n in numbers:  
        if n%2 == 0:  
            total +=n  
    return total
```

MODEL ANSWER

```
def sum_even_numbers(numbers):  
    total = 0  
    for num in numbers:  
        if num % 2 == 0:  
            total += num  
    return total  
  
# Example usage:  
# print(sum_even_numbers([1, 2, 3, 4, 5, 6])) # 12  
  
# Handles an empty list by returning 0.  
# Ignores odd numbers and sums only values divisible by 2.  
# Works with negative even numbers as well, since -4 % 2 == 0.
```

HOW MARKS WERE AWARDED

Function definition, loop through the list, check if number is even, return sum.

The function correctly defines the required logic to sum even numbers, fulfilling all rubric criteria.

QUESTION 29 · SUBJECTIVE**3/5 marks**

What are the different logging levels in Python and when would you use each?



YOUR ANSWER

Python have five logging levels:
DEBUG – for details debugging information
INFO – for normal program event
WARNING – for possible problems
ERROR – for errors
CRITICAL – for serious errors that may stop the program

MODEL ANSWER

Python's logging module has five standard logging levels, each used for a different kind of message:

1. **DEBUG:** Used for detailed diagnostic information, mainly useful while developing or troubleshooting. For example, logging variable values, loop progress, or function entry/exit.
2. **INFO:** Used to confirm that things are working as expected. For example, logging that a service started successfully, a file was loaded, or a user logged in.
3. **WARNING:** Used for something unexpected or a potential problem, but the program can still continue. For example, a deprecated function is being used, or a retry is about to happen because a request failed once.
4. **ERROR:** Used when a more serious problem occurs and a specific operation fails. The application may still run, but part of it did not work correctly. For example, failing to open a file, or a database query error.
5. **CRITICAL:** Used for very severe errors that may stop the program or make it unable to continue properly. For example, complete loss of database connection, or failure to start a required service.

In practice, **DEBUG** is for development details, **INFO** for normal application events, **WARNING** for recoverable issues, **ERROR** for operation failures, and **CRITICAL** for fatal problems.

HOW MARKS WERE AWARDED

Key points: DEBUG, INFO, WARNING, ERROR, CRITICAL; examples of when to use each level.

The answer correctly identifies the logging levels but lacks examples of when to use each level.

QUESTION 30 · CODE

4/5 marks

Create a function that takes a string and returns it in reverse order.

YOUR ANSWER

```
def str_reverse(text)
    return text[::-1]
```

MODEL ANSWER

```
def reverse_string(s):
    # Return the string reversed using slicing
    return s[::-1]
```

```
# Example usage:
# print(reverse_string("hello")) # "olleh"
```

Edge cases handled: an empty string returns an empty string. A one-character string returns the same character. The function works for spaces, punctuation, and any Unicode characters because slicing reverses the full string.

HOW MARKS WERE AWARDED

Function definition, use slicing or loop, return reversed string.

The function is correctly defined and uses slicing, but is missing a colon after the function definition.

QUESTION 31 · SUBJECTIVE

5/5 marks



Write a function that takes a list of numbers and returns the sum of all even numbers.

YOUR ANSWER

```
def sum_even(numbers):  
    total = 0  
    for i in numbers:  
        if i % 2 == 0:  
            total += i  
    return total
```

MODEL ANSWER

```
def sum_even_numbers(numbers):  
    total = 0  
    for num in numbers:  
        if num % 2 == 0:  
            total += num  
    return total
```

Example:

```
# print(sum_even_numbers([1, 2, 3, 4, 5, 6])) # Output: 12
```

This function loops through each number in the list, checks whether it is even using a conditional, and adds it to the running total if it is divisible by 2. Finally, it returns the total sum.

It handles an empty list by returning 0, and it works correctly when there are no even numbers, since the total remains 0.

It also works with negative even numbers because the modulo check still identifies them as even.

HOW MARKS WERE AWARDED

Key points: correct use of loops, conditionals, and return statement.



The function correctly implements a loop, uses a conditional to check for even numbers, and includes a proper return statement.

QUESTION 32 · CODE

5/5 marks

Implement a function that raises a `ValueError` if the input is not an integer.

YOUR ANSWER

```
def check_integer(value):  
    if not isinstance(value, int):  
        raise ValueError("Input must be an integer")  
    return value
```

MODEL ANSWER

```
def validate_integer(value):  
    # Check that the input is an integer  
    if not isinstance(value, int):  
        raise ValueError("Input must be an integer")  
    return value  
  
# Edge cases handled:  
# 1) Non-integer inputs such as strings, floats, lists, and None raise ValueError.  
# 2) Valid integers are accepted and returned unchanged.  
# 3) The type check is explicit, so only int values pass.
```

HOW MARKS WERE AWARDED

Function definition, use `isinstance` to check type, raise `ValueError`.

The function correctly defines the input check, uses `isinstance` for type verification, and raises a `ValueError` as required.

QUESTION 33 · CODE

3/5 marks



Write a Python function that takes a string and returns the string reversed. Handle any exceptions that might occur.

YOUR ANSWER

```
def string_reverse(text):  
    try:  
        return text[::-1]  
    except Exception as e:  
        return str(e)
```

MODEL ANSWER

```
def reverse_string(s):  
    try:  
        # Ensure the input is actually a string  
        if not isinstance(s, str):  
            raise TypeError("Input must be a string")  
        # Reverse using slicing  
        return s[::-1]  
    except TypeError as e:  
        return f"Error: {e}"
```

```
# Example usage  
print(reverse_string("hello"))  
print(reverse_string(123))
```

This handles the main edge case of a non-string input by raising and catching a `TypeError`. It also avoids crashing the program and returns a clear error message. For valid strings, it correctly returns the reversed string using slicing.

HOW MARKS WERE AWARDED

Expected approach: use slicing to reverse the string; handle `TypeError` if input is not a string.

The function correctly reverses the string but does not specifically handle `TypeError` for non-string inputs.



QUESTION 34 · SCENARIO

5/5 marks

You have a list of temperatures in Celsius. Write a function that converts them to Fahrenheit and logs the conversion process.

YOUR ANSWER

```
import logging

logging.basicConfig(level=logging.INFO)

def convert(celsius):
    result = []
    for c in celsius:
        f = (c * 9/5) + 32
        result.append(f)
        logging.info(f"{c}C = {f}F" )
    return result
```

MODEL ANSWER

```
def convert_and_log_temperatures(celsius_list):
    """Convert a list of Celsius temperatures to Fahrenheit and log each step."""
    fahrenheit_list = []
    for c in celsius_list:
        f = (c * 9 / 5) + 32
        print(f"Converting {c}°C to Fahrenheit: {f}°F")
        fahrenheit_list.append(f)
    return fahrenheit_list
```

Example usage

```
temps_c = [0, 25, 100, -10]
temps_f = convert_and_log_temperatures(temps_c)
print("Converted list:", temps_f)
```



This function takes a list of Celsius temperatures as input and returns a new list of Fahrenheit values. It uses the correct conversion formula, $\text{Fahrenheit} = (\text{Celsius} \times 9/5) + 32$, for each item in the list. It also logs every conversion with a clear message showing the original Celsius value and the converted Fahrenheit value.

It handles an empty list by returning an empty list and printing nothing. It also works with negative temperatures and decimal values because the formula is applied to each numeric element individually.

HOW MARKS WERE AWARDED

Function definition, conversion formula, logging each conversion.

The function is well-defined, correctly implements the conversion formula, and logs each conversion as required.

QUESTION 35 · CODE

5/5 marks

Create a function that accepts two numbers and returns their division. Ensure to handle division by zero exceptions.

YOUR ANSWER

```
def divide(a,b):  
    try:  
        return a/b  
    except ZeroDivisionError:  
        return "Cannot divide by zero"
```

MODEL ANSWER

```
def divide_numbers(a, b):  
    # Returns the result of dividing a by b, handling division by zero safely  
    try:  
        return a / b  
    except ZeroDivisionError:
```



```
return "Cannot divide by zero"
```

```
# Example usage
print(divide_numbers(10, 2))
print(divide_numbers(10, 0))
```

This handles the normal case where the divisor is non-zero and returns the correct quotient.
If the second number is 0, it catches ZeroDivisionError and returns a clear message instead of crashing.
It also works with integers and floats, since Python division supports both.

HOW MARKS WERE AWARDED

Expected approach: use try-except to catch ZeroDivisionError; return appropriate message or value.

The function correctly implements division with exception handling for division by zero.

QUESTION 36 · SCENARIO

4/5 marks

Describe how you would handle an exception that occurs when reading from a file in Python.

YOUR ANSWER

I would use try and except to handle errors while reading a file. for example, FileNotFoundError can be handled when the file does not exist.

```
try:
    with open("data.txt", "r") as file:
        data = file.read()
except FileNotFoundError:
    print("File not found")
```

MODEL ANSWER

I would wrap the file-reading code in a try-except block so the program can handle failures gracefully instead of crashing.
For example, I would first use try to open and read the file, then catch specific exceptions such as FileNotFoundError if the



file does not exist, `PermissionError` if the program does not have access, and possibly `OSError` for other I/O-related problems.

Inside each `except` block, I would log a clear error message that includes the file name and a short description of the problem. Logging is better than printing because it records the error for debugging and support later. If needed, I would also re-raise the exception after logging it, or return a safe default value, depending on how important the file data is to the program.

This approach makes the code more robust because it handles known errors in a controlled way, gives useful diagnostics through logging, and allows the program to continue or fail cleanly based on the situation.

HOW MARKS WERE AWARDED

Use of `try-except`, handling specific exceptions, logging error messages.

The answer correctly uses `try-except` and handles a specific exception, but it lacks logging error messages.

